

Governed Support Automation Platform

Living Vision and Working Reference

Document role	Authoritative living reference for the platform, product vision, design doctrine, and architect notes
Status	Active v1.5; one of two living project documents
Owner	Akbar Aman
Date	July 27, 2026
First reference implementation	MOHID Support Automation
Human control plane	SON - Support Operations Navigator
Commercial brand	Intentionally deferred; this paper uses the product category name
Companion document	MOHID Problem and First Implementation Living Specification

Vision in one sentence

The platform automatically resolves support requests when evidence and policy permit, routes uncertain or restricted cases to the appropriate humans, and preserves one auditable record of every AI action, human decision, tool execution, delay, and final outcome.

Automate what is safe. Route what requires judgment. Audit the full lifecycle.

Document Control

Purpose	Define the durable product thesis, category, operating model, human role, governance principles, platform boundaries, project lineage, architect notes, and long-term direction.
Audience	Product owner, engineers, reviewers, implementation partners, business stakeholders, and future maintainers.
Scope	Platform-level vision, operating model, reusable architectural boundaries, design doctrine, and working decision frontier. MOHID-specific schemas, services, milestones, and implementation tasks belong in the companion living specification.
Ground truth	This document and the companion MOHID living specification are the only active project documents. They supersede the earlier v0.1, v0.2, Vision v1.0, authoring handoffs, and chat-generated drafts.
Version rule	This is a living document with stable filename and numbered internal revisions. Material corrections or strategic changes must update the revision history and affected decisions rather than create another competing paper.
Provider stance	The product is provider-agnostic at the contract and vision layers. Individual deployments may deliberately adopt a cloud or model provider when that creates clear operational value.

How this paper should be used

Use this document to answer *what the platform is becoming, why it exists, which boundaries are durable, and which strategic questions remain open*. Do not use it as the MOHID implementation backlog. The companion living specification answers *what is being built for MOHID now, in what order, and how it will be validated*.

Revision History

Version	Date	Status	Summary
0.1	July 2026	Superseded	Broad evidence-backed support resolution platform specification.
0.2	July 2026	Superseded	Engineering-focused governed support OS specification.
1.0	July 2026	Superseded base-line	Concise product vision centered on configurable automation, human oversight, organizational routing, SON, and unified AI-human accountability.
1.1	July 27, 2026	Superseded base-line	Consolidated project lineage, established the two-document source-of-truth model, and added the architect decision frontier.
1.2	July 27, 2026	Superseded base-line	Removed repeated provenance clarification and locked the first implementation to company-approved providers, test-driven delivery, and provider-neutral product contracts.
1.3	July 27, 2026	Superseded base-line	Defined runtime/application ownership, immutable run-version binding, and the non-bypassable platform policy floor.
1.4	July 27, 2026	Superseded base-line	Added the knowledge-compiler boundary, subsystem-level failure attribution, and the bounded proof-of-concept versus proof-of-value doctrine.
1.5	July 27, 2026	Active living reference	Clarifies that external APIs supply underlying capabilities, MCP is an optional connector protocol, semantic provider contracts remain authoritative, and knowledge reads, writes, and operational actions retain distinct governance.

Contents

Document Control	i
1 Executive Thesis	1
2 Project Lineage, Authorship, and Document Boundary	1
2.1 Independent Product and IP	1
2.2 Product and Technical Predecessor	2
2.3 The Two-Document Rule	2
3 Origin and the Problem Being Solved	2
4 Market Context: Why This Product Exists	3
5 Product Category and Long-Term Vision	4
6 Core Operating Model	5
6.1 Resolution, Dispatch, and Routing	6
7 Configurable Autonomy and Organizational Fit	6
8 Human Role and Unified Accountability	7
8.1 Dual Accountability Model	7
9 SON: The Support Operations Navigator	8
10 Platform Boundary and Provider Independence	9
10.1 Connector Protocols and MCP Fit	10
10.2 Knowledge-Compiler Contract	11
10.3 Runtime–Application Contract	11
11 Design Doctrine	12
12 Architect Notes and Decision Frontier	14
12.1 Locked Platform Decisions	14
12.2 Current Working Positions	14
12.3 Open Strategic Decisions	15
13 Evaluation and Responsible Expansion	15
14 MOHID as the First Reference Implementation	16
15 Explicit Non-Goals	17
16 Closing Vision	17
References	19

1 Executive Thesis

Support organizations do not need another unconstrained chatbot. They need a dependable operating layer that can convert approved organizational knowledge into correct action, determine when automation is allowed, route exceptions to the appropriate humans, and explain how each case reached its outcome.

The platform therefore treats support as a governed lifecycle rather than a single model response. Every request becomes a support case. The case may be resolved automatically, prepared for rapid human approval, clarified, escalated, rejected, or closed without action. The organization decides the permitted level of autonomy for each category, product, customer tier, team, and action.

Core product thesis

A support operation cannot be governed by auditing AI alone. It must also preserve the human decisions that approve, change, delay, override, route, or reject AI-assisted resolutions. The product creates one case-level record across both.

The primary business outcome is faster and more consistent resolution. Governance, evidence, verification, and auditability are the mechanisms that make automation acceptable and expandable. Human review is neither a temporary defect nor a mandatory step for every case. It is a configurable control that can range from universal pre-send approval to targeted exception handling and post-resolution sampling.

The product is designed to support organizations that operate in different ways:

- one centralized support team;
- small teams divided by product or functional area;
- tiered Level 1, Level 2, and specialist escalation;
- regional or customer-tier routing;
- mostly human review with selective automation; or
- mostly automatic resolution with humans concentrated on exceptions.

The first reference implementation is MOHID. The long-term platform is broader, but it must emerge from demonstrated operational need rather than speculative generalization.

2 Project Lineage, Authorship, and Document Boundary

2.1 Independent Product and IP

The platform is authored and directed by Akbar Aman as an independent product and systems architecture. MOHID is the first reference application and commercial problem being solved; it does not define the full platform boundary. Reusable contracts, runtime behavior, governance, evaluation methods, and platform design remain distinct from client-specific documentation, terminology, credentials, policies, integrations, and deployment choices.

2.2 Product and Technical Predecessor

Artifact	What it is	Relationship to this platform
legacy-prototype-v1	The earlier standalone MOHID support implementation: FastAPI, FAISS, Mailgun, Render, and direct model-provider integrations.	Supplies a technical baseline, reusable fixtures, email lessons, and regression cases. It does not define the new product architecture.
Governed Support Automation Platform	The independently authored product defined in this document.	Owns the durable case model, evidence and decision contracts, governance, provider boundaries, SON, audit model, evaluation doctrine, and long-term product direction.

Confidentiality and non-inheritance boundary

One high-level influence was ASTRA, an enterprise managed AWS agent runtime encountered through AHEAD; it informed general runtime/application separation and governance thinking but is neither product lineage nor a dependency. Confidential work artifacts and proprietary implementation details must not be copied into client-facing or public work.

2.3 The Two-Document Rule

This project has exactly two living sources of truth:

1. **This platform living reference** owns the long-term product vision, reusable system boundaries, design doctrine, authorship, and architect decision frontier.
2. **The MOHID living specification** owns the client problem, contractual and operational facts, the previous and target technology stacks, first-implementation requirements, evaluation, rollout, and build plan.

The v0.1 platform paper, v0.2 OS paper, Vision v1.0, authoring handoffs, research reports, notebook conversations, and GitHub repository documentation are retained as historical evidence. They are not separately maintained. A new finding becomes authoritative only when it is reconciled into one of these two living documents.

3 Origin and the Problem Being Solved

The project originated from a practical support automation problem. MOHID possesses strong owner-written documentation. The missing capability was not simply access to answers; it was the reliable conversion of that knowledge into support resolution across email, direct responses, ticket creation, forms, API actions, and escalation paths.

Direct stakeholder discussions and earlier implementation work established a stable operational pattern:

1. a customer inquiry arrives through a support channel;
2. the request is interpreted and matched to approved documentation;

3. routine questions may be answered directly;
4. some requests require a ticket, prefilled form, external API, or routed escalation;
5. higher-level support handles cases that exceed the first-line scope; and
6. feedback, reopening, and final resolution matter after a response is sent.

The broader business pain is not limited to answer generation. It includes:

- inconsistent reviewer judgment and response quality;
- repetitive investigation even when authoritative documentation exists;
- tickets that remain open longer than their evidence or complexity justifies;
- limited visibility into where time was spent and which event, state, or dependency extended a case;
- senior or owner-level experts becoming bottlenecks for routine issues;
- weak transfer of institutional knowledge to less experienced reviewers; and
- workforce resistance when AI is framed only as replacement.

This is partly a management problem, and software cannot determine employee motive or replace leadership. The platform's role is narrower and defensible: it removes ambiguity from the case lifecycle by recording what information was available, when the case became resolution-ready, what the system proposed, what the human did, and how the final outcome performed.

Observable facts, not inferred motives

The platform may identify queue time, evidence availability, clarification cycles, handoffs, edits, overrides, and reopenings. It must not label a reviewer dishonest, lazy, or intentionally obstructive. Personnel judgments remain human responsibilities.

4 Market Context: Why This Product Exists

AI-assisted support is already an established market category. Zendesk currently offers AI-generated replies, actions, and instructions that agents can review, edit, approve, or dismiss, with sources shown when available.[1] Intercom's Fin can answer from multiple knowledge sources, resolve customer conversations, and hand unresolved cases to humans or external support tools.[2, 3] Salesforce Agentforce likewise positions autonomous service agents and assistive service tools around common-case resolution, grounding, and human support workflows.[4]

The market therefore validates several assumptions:

- organizations want automated resolution, not only drafting;
- support knowledge can be used to ground AI answers;
- AI-to-human handoff is a required operating pattern;
- agents benefit from suggested actions and prepared case context; and

- managers need performance and conversation visibility.

The product does not claim that AI support, human handoff, citations, or ticket analytics are new. Its reason to exist is the need for a more explicit and configurable resolution-control layer.

The remaining gap

The platform unifies the evidence, AI recommendation, dispatch authority, organizational routing, human intervention, tool execution, timing, and final outcome of a support case. Its unit of accountability is the complete case lifecycle, not merely the generated answer or ticket log.

The intended differentiation rests on the combination of:

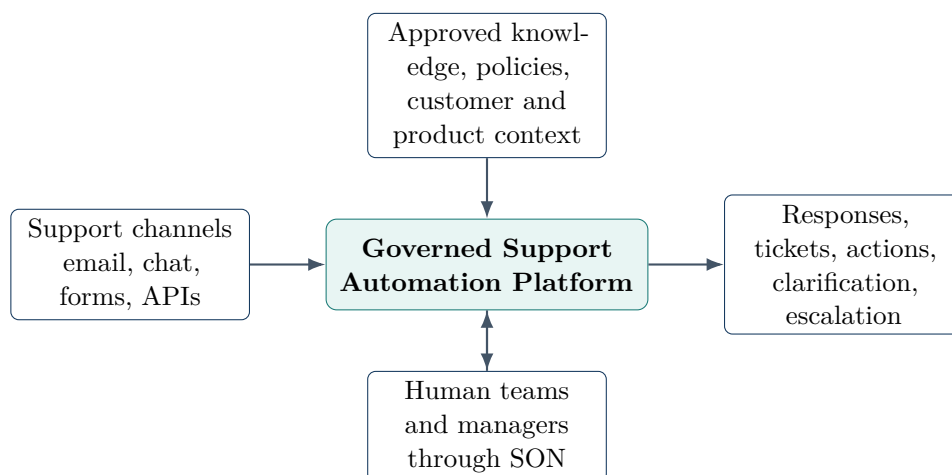
- evidence-to-claim and evidence-to-procedure traceability;
- explicit separation of resolution, dispatch, and routing;
- bounded organizational configuration rather than one fixed queue model;
- policy-controlled automatic tool execution;
- structured human approvals, edits, overrides, and reasons;
- decomposition of case time into system, queue, reviewer, customer, and external dependency intervals;
- post-resolution sampling, feedback, and reopening analysis; and
- replayable case histories across model, policy, or knowledge changes.

The product must still prove that these capabilities create enough incremental value beyond established platforms. The MOHID implementation is the first evidence-generating deployment, not proof by itself.

5 Product Category and Long-Term Vision

The product category is **configurable governed support automation**.

It sits between organizational knowledge, support channels, business tools, and human teams. It may complement an existing help desk, act as a specialized operating layer, or eventually become the primary resolution surface for a bounded support function. It is not required to replace the organization's ticketing, CRM, or communication systems.



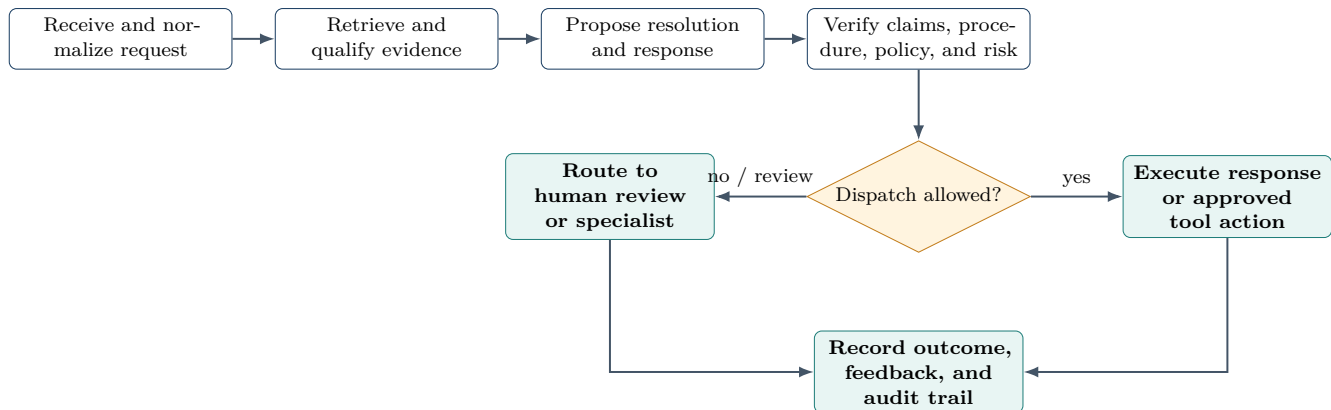
The long-term system should allow an organization to declare:

- which knowledge sources are authoritative;
- which categories may be resolved automatically;
- which actions require approval or are prohibited;
- which team owns each type of case;
- who may approve, override, or expand automation;
- how quality is sampled and evaluated; and
- what evidence is required to defend a final outcome.

The commercial name remains intentionally separate from this vision. The product category and architecture must remain coherent even if branding changes.

6 Core Operating Model

Every request enters one governed lifecycle. The same lifecycle supports direct automation, human review, and escalation.

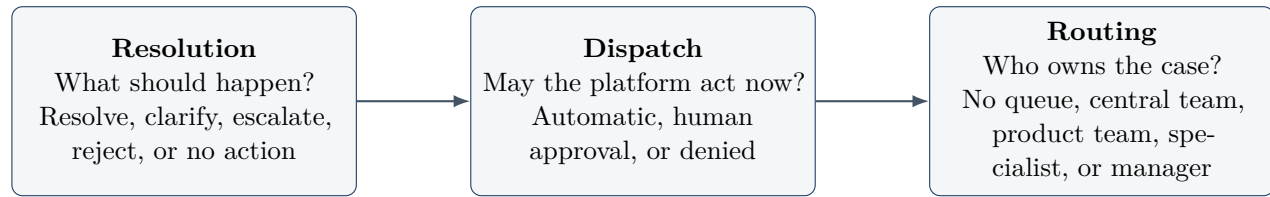


The system should produce typed outcomes rather than treating every request as an answer-generation problem:

Resolve	Sufficient evidence and authority exist to answer or perform the supported action.
Clarify	Missing information or ambiguity prevents a safe resolution.
Escalate	The case requires higher authority, account access, exception handling, or specialist judgment.
Reject	The request is prohibited, invalid, abusive, or outside the supported boundary.
No action	The event is a duplicate, notification, or other item that requires no outbound response.

A proposed resolution is not the same as permission to act. The platform must keep three decision planes separate.

6.1 Resolution, Dispatch, and Routing



This separation permits outcomes such as:

- **Resolve + automatic + no team:** a routine, fully supported answer is sent and closed.
- **Resolve + human approval + billing team:** the answer is prepared but requires review.
- **Escalate + automatic routing + specialist team:** no answer is sent, but the correct case is created and assigned.
- **Reject + denied + manager visibility:** the request is blocked and recorded without unsafe action.

7 Configurable Autonomy and Organizational Fit

The product must not impose a universal support structure. It provides bounded primitives that an organization combines into its operating model.

Centralized model	All requests enter one queue, with broad reviewers and specialist escalation.
Product-team model	Requests route to teams responsible for specific products or modules.
Functional model	Billing, technical, account, compliance, and other functions own distinct queues.
Tiered model	Automation acts as Tier 0; general reviewers handle Tier 1; specialists and managers handle higher tiers.
Hybrid model	Routine categories auto-resolve, medium-risk cases require approval, and privileged actions require specialist or dual approval.

Configuration should use a fixed vocabulary of auditable conditions rather than arbitrary per-client code. Typical policy inputs include:

- support category and product;
- evidence sufficiency and conflict status;
- customer tier, region, language, or channel;
- data sensitivity and action privilege;
- verification results and known risk classes;
- tool type and side-effect level;
- reviewer authority and team ownership; and
- rollout stage or required sampling rate.

An organization may begin with human approval for every outbound response, then expand automation category by category. The platform must also support the reverse: an organization may pause automation globally, narrow it to selected categories, or route all cases to review during incidents or knowledge changes.

Autonomy is a policy setting, not a product identity

The same platform may operate at 0 percent automatic dispatch or at a high automation rate. The invariant is not how much the system automates; it is that every autonomous action is evidence-backed, policy-authorized, verified, and reconstructable.

8 Human Role and Unified Accountability

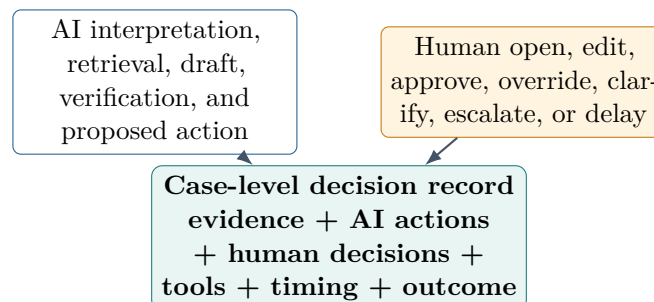
The platform is designed around a practical labor model: reliable reviewers should be able to supervise more support volume because the system performs repetitive investigation, evidence collection, drafting, and preliminary verification.

The intended human role is to:

- exercise judgment on ambiguous, sensitive, or exceptional cases;
- approve, edit, reject, or reroute prepared resolutions;
- challenge incorrect evidence or model reasoning;
- handle privileged actions and policy exceptions;
- review samples of automated cases;
- diagnose recurring failures and knowledge gaps; and
- govern the expansion or contraction of automation.

Human review must be meaningful. A reviewer should not become a ceremonial approval button for an answer they cannot evaluate. Sensitive cases should expose the evidence, verification results, unresolved conflicts, and policy rationale needed for real judgment. Overrides should capture a lightweight reason code without creating unnecessary administrative burden.

8.1 Dual Accountability Model



The platform should record, at minimum:

AI activity	interpretation, retrieval queries, source spans, proposed decision, generated claims, confidence, verification, policy outcomes, and tool intent.
Human activity	assignment, open time, edits, approval, rejection, override reason, clarification request, escalation, and final dispatch.
Tool activity	validation, authorization, idempotency, execution result, retry, failure, and external receipt.
Case outcome	total time, queue time, active handling time, customer wait, external dependency, handoffs, reopening, feedback, and final disposition.

The design must avoid turning accountability into simplistic surveillance. Speed metrics must be paired with complexity, correctness, reopening, customer outcome, and external dependencies. Individual-level analytics should be restricted to authorized roles and should not be the initial product surface.

NIST's AI Risk Management Framework emphasizes accountability, transparency, documentation, and human-AI interaction across the system lifecycle.[6] The platform adopts that socio-technical view: trustworthy resolution depends on the operating process around the model, not only the model itself.

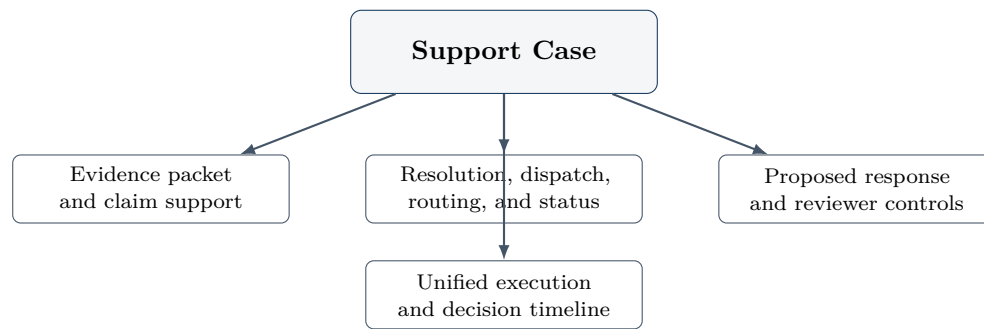
9 SON: The Support Operations Navigator

SON is the human-facing control plane for the platform. It is not merely an administrative dashboard and it is not a chatbot interface.

SON answers the operational questions that matter:

- What happened to this support request?
- Where is the case now?
- What evidence supports the proposed outcome?
- Did verification or policy block automation?
- Does a human need to intervene?
- Which team owns the next action?
- Why is the case still open?
- What did the AI propose and what did the human change?
- Did the final outcome actually work?

The primary human-visible object is a **Support Case**. A case may contain one or more internal request runs, model calls, retrieval operations, tool calls, and human decisions. Engineers may drill into those details; operators should first see the case lifecycle and current required action.



SON should eventually support three modes:

1. **Human review:** cases awaiting approval, correction, clarification, or escalation.
2. **Automation supervision:** auto-resolved cases, sampled cases, failures, policy blocks, and reopenings.
3. **Management oversight:** queue health, delay decomposition, routing quality, recurring topics, knowledge gaps, and operational outcomes.

The interface should be configurable to the organization's support structure while retaining the same case and audit model underneath.

10 Platform Boundary and Provider Independence

The product's durable value must reside in its own contracts and operating logic rather than in a single model, cloud, vector database, or agent framework.

The portable core includes:

- support-case and request-run models;
- evidence-packet and source-provenance contracts;
- resolution, dispatch, and routing decisions;
- policy and permission logic;
- tool-intent and execution contracts;
- human-decision and audit-event schemas;
- evaluation and replay definitions; and
- SON's case-centric interaction model.

Provider-specific implementations may supply:

- language models and embedding models;
- vector search and reranking;
- object storage and relational persistence;
- identity, secrets, logging, and cloud hosting;

- email, ticketing, CRM, and business-system connectors; and
- managed agent or workflow infrastructure.

The previous MOHID implementation is referenced only as a technology baseline and a source of regression fixtures. The first production-oriented implementation should adopt MOHID/DeenTek-approved providers at the edges while keeping the platform's case, evidence, decision, policy, audit, and tool contracts independent of those providers.

Portable core, deliberate deployment

The platform should remain provider-neutral where switching is strategically valuable. A specific deployment may intentionally adopt GCP, AWS, or another vendor when customer alignment, security, cost, or operational simplicity justifies it. Multi-cloud parity is not required on day one.

Provider neutrality must be practical rather than theatrical. The system should define narrow adapters at meaningful boundaries, implement the provider actually needed, and prove portability later with a second implementation. It should not build duplicate integrations before the product works.

10.1 Connector Protocols and MCP Fit

An external API, SDK, database, export, webhook, crawler target, or other supported interface supplies an underlying capability. The Model Context Protocol (MCP) is an optional AI-facing protocol through which a server may expose resources or tools to a client; it does not replace the underlying system or define this platform's business semantics.^[5] The durable relationship is:

**underlying interface → direct adapter or MCP server → semantic provider contract →
governed runtime**

The runtime therefore understands support capabilities rather than “an MCP server.” REST, SDK, database, export, webhook, crawler, MCP, and fake implementations remain replaceable beneath the same contracts.

Capability class	Semantic boundary	Permanent governance rule
Batch knowledge synchronization	A KnowledgeSourceProvider discovers and fetches approved sources or lists changes from a checkpoint.	Any adapter or MCP wrapper may improve acquisition, but the result still passes through raw snapshotting, canonicalization, validation, approval, and corpus publication.
Live knowledge querying	A KnowledgeQueryProvider searches approved knowledge or retrieves an exact document with source metadata and version information.	Before a result supports a consequential decision, it is normalized, snapshotted, hashed, timestamped, assigned stable evidence spans, bound to the case and run, and retained for audit and replay. A changing transient response is not evidence.
Knowledge-base writing	A separately permissioned write capability proposes drafts, updates, deprecation, or publication.	A detected gap produces a proposed change, deterministic validation, human knowledge-owner review, separately authorized publication, a provider receipt, compiler synchronization, evaluation, and a new corpus version. A model cannot directly rewrite authoritative knowledge.
Operational business actions	A ToolExecutor performs an already authorized ticket, account, receipt, routing, or other bounded action.	Operational credentials, scopes, policy, receipts, and failure handling remain distinct from knowledge access. A broad connector may not collapse unrelated permissions into one authority.

These classes define the platform's capability and risk vocabulary; they do not imply that every deployment implements every capability. Semantic permissions may be granular—for example, `knowledge.search`, `knowledge.sync`, `knowledge.publish`, `ticket.route`, or `operation.execute`—but connector authentication alone never grants them.

Protocol is not policy

Whether a capability arrives through a direct API adapter or MCP, the governed runtime remains authoritative for tenant, application, actor, environment, source authority, case/run state, requested capability, granted scope, risk, evidence, approval, policy version, idempotency, rollout stage, and kill-switch checks.

10.2 Knowledge-Compiler Contract

Authoritative support knowledge must not be flattened directly into arbitrary model context. The platform treats knowledge preparation as a deterministic compilation boundary:

approved source → raw snapshot → canonical block tree

canonical block tree ⇒ human-readable derivative + search projections

The canonical structured representation preserves source semantics such as headings, procedures, prerequisites, warnings, tables, links, and stable locators. Markdown or another review-friendly format is generated from that representation for inspection and change review; it is not the sole source of truth. Search chunks, lexical fields, embeddings, and any parent-context links are separate, reproducible projections of the same blocks.

The parser and scaffolder should be deterministic code. Retrieval strategies such as section blocks, procedure blocks, parent-child expansion, hybrid search, or reranking remain benchmarked implementation choices rather than platform doctrine. A tenant package identifies approved sources and evidence requirements, while the runtime owns publication integrity, version binding, traceability, and rollback.

10.3 Runtime–Application Contract

Deployment equation

Shared governed runtime + versioned tenant application package = deployed support application.

The runtime owns the canonical lifecycle—intake, evidence, decision, verification, policy, dispatch or routing, and outcome—and the controls that make it safe and reconstructable. A tenant application package declares bounded behavior inside that lifecycle; it does not supply arbitrary executable code, replace the workflow, or bypass mandatory gates.

Runtime-owned enforcement	Tenant resolution; case and run state; provider invocation; retrieval and verification mechanics; minimum policy and permissions; tool authorization and idempotency; secrets; audit; retries; recovery; and kill switches.
Application-declared behavior	Approved knowledge sources; terminology and taxonomy; routing; templates and branding; category-level autonomy; permitted capabilities within the platform floor; evidence requirements; and evaluation cases.
Deployment bindings	Versioned references to approved model, embedding, storage, identity, and channel profiles. Provider SDKs, credentials, and secret material remain outside the application package.

Identity remains explicit as `tenant_id`, `application_id`, and `environment`. Before a run starts, the runtime validates and hydrates the selected package into an immutable configuration snapshot. The run pins the runtime, application-package, workflow, corpus, policy, model-profile, and adapter versions. A configuration change applies only to a later run; a missing or incompatible version fails closed.

Platform floor, tenant choice

A tenant may narrow automation, require more review, restrict tools, or demand stronger evidence. It may not disable tenant isolation, required audit, deterministic authorization, mandatory verification, recovery controls, or the global kill switch.

11 Design Doctrine

The following principles are the permanent baseline for product evolution.

1. Evidence before consequential action

The system must identify approved support for a response or action before it can proceed. Retrieval similarity alone is not evidence sufficiency.

2. Typed decisions over unconstrained agent behavior

The platform's primary outputs are structured decisions, not free-form model behavior. Natural-language responses are bounded artifacts within a governed case.

3. Resolution, dispatch, and routing remain separate

Knowing what should happen does not imply permission to execute it or determine who owns it.

4. Automatic execution fails closed

Missing evidence, failed verification, policy uncertainty, tool failure, or unavailable authority must reduce autonomy rather than encourage guessing.

5. Models propose; deterministic systems authorize and execute

Language models may identify intent and propose actions. Deterministic runtime code must validate permissions, required fields, idempotency, side effects, and transaction success. Authentication to a direct connector or MCP server cannot substitute for this authorization.

6. Human intervention is a valid outcome

Clarification and escalation are successful behavior when automation lacks evidence or authority. Automation rate must never be optimized by suppressing appropriate handoff.

7. Audit the complete socio-technical process

The system must reconstruct the evidence, AI activity, human decisions, tool results, timing, and outcome of consequential cases.

8. Observe behavior without inferring intent

The platform may report objective timing patterns and policy-defined service-level exceptions. It must not make disciplinary or motivational judgments about employees.

9. Expand autonomy through measured evidence

New categories and tools become automatic only after replay, shadow testing, supervised use, and defined acceptance thresholds demonstrate acceptable performance.

10. Generalize from proven need

The MOHID implementation should produce reusable primitives. The platform must not pre-build arbitrary enterprise complexity before a real deployment requires it.

11. Tenant knowledge, policy, and data remain isolated

Each organization controls its approved sources, terminology, routing, permissions, integrations, evaluations, and retention requirements.

12. The case is the product's unit of truth

Model runs, messages, and ticket events are subordinate to the Support Case, which represents the complete operational lifecycle and final outcome.

13. Preserve source meaning before optimizing retrieval

Raw sources, canonical structure, human-readable review artifacts, and search projections remain distinct. Retrieval optimization may not destroy the instructions, ordering, warnings, or provenance needed to defend an answer.

14. Diagnose failures at the boundary where they occur

Knowledge, retrieval, model availability, policy, channel delivery, persistence, and human workflow are measured separately. A provider outage must not be labeled an answer-quality failure, and a successful send must not be labeled a correct resolution.

15. Separate technical proof from value proof

One bounded end-to-end slice must first prove that the governed lifecycle works. The same real cases must then show better correctness, safer abstention, or lower human effort than the baseline before platform breadth is justified.

12 Architect Notes and Decision Frontier

This section is the maintained bridge between long-term vision and current reasoning. It records which conclusions are locked, which positions guide near-term work, and which questions must remain open until implementation evidence exists.

12.1 Locked Platform Decisions

- The product category is governed support resolution, not generic chat or arbitrary agent automation.
- The Support Case is the durable unit of work and accountability.
- Evidence, resolution, verification, dispatch, routing, tool execution, human action, and outcome share one reconstructable lifecycle.
- Resolution, dispatch, and routing remain separate typed decisions.
- SON is the human control plane for review, supervision, and operational visibility.
- Automatic resolution is a supported outcome, but eligibility is policy- and evidence-controlled.
- Provider neutrality belongs in product contracts; equal multi-cloud implementation is not a near-term goal.
- Provider contracts express semantic capabilities and remain protocol-neutral; MCP is an optional adapter mechanism, not a permanent platform dependency or domain abstraction.
- A deployed support application is a versioned tenant package executed by the shared governed runtime; tenant behavior cannot replace the canonical lifecycle or weaken the platform policy floor.
- MOHID is the first application and evidence source, not the permanent limit of the product.

12.2 Current Working Positions

- Begin with one explicit, testable workflow or state machine rather than a free-form multi-agent swarm.
- Separate application-declared knowledge, terminology, templates, and policy from runtime-owned execution, verification, audit, and side-effect controls.
- Build the first vertical slice on the company-approved MOHID/DeenTek stack rather than on a temporary alternate production stack.
- Drive implementation from failing acceptance, contract, integration, and replay tests before adding production behavior.
- Add adapters only at meaningful provider boundaries and implement one approved provider per boundary; local fakes support tests but do not become an alternate deployment architecture.

- Choose direct REST, SDK, database, export, crawler, or MCP integration only after the real provider capability and reuse need are known; a generalized MCP layer is not a prerequisite for the first proof.
- Compile approved knowledge into raw snapshots, canonical structured blocks, generated review artifacts, and versioned search projections rather than treating Markdown or embeddings as the sole source of truth.
- Select chunking, context expansion, hybrid retrieval, and reranking through replay evidence; no named retrieval pattern is mandatory without measured advantage.
- Extract reusable primitives only after the MOHID vertical slice demonstrates that they solve a recurring problem.
- Preserve the earlier GitHub implementation as a technical baseline and fixture source; port only reviewed assets that satisfy the new contracts and tests.

12.3 Open Strategic Decisions

Commercial identity	Final product and company-facing brand, packaging, pricing, and ownership presentation.
Second reference organization	Which deployment proves that the platform boundary is genuinely reusable rather than MOHID-specific.
Runtime implementation	Exact orchestration library, skill model, deployment control surface, and degree of managed-service adoption after V1 evidence.
Provider portfolio	When a second model, vector, cloud, email, or knowledge adapter is justified, and when an MCP wrapper creates measured reuse or interoperability value beyond a direct integration.
Platform versus integration	Which capabilities remain native and which should be delegated to existing help desks, CRMs, identity systems, or communication platforms.
Post-MOHID roadmap	Which platform capabilities become product commitments after contract closure and measured pilot results.

Decision discipline

Open questions are not missing architecture. They are intentionally deferred commitments. The platform should close them through deployment evidence, customer need, cost, security, and maintainability rather than through speculative breadth.

13 Evaluation and Responsible Expansion

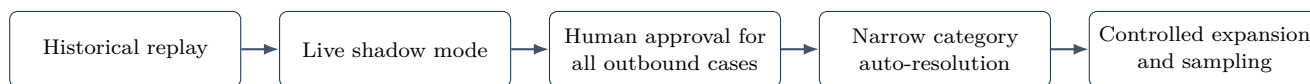
The platform's vision depends on measurable operational improvement. It should not declare success because a model generates plausible responses.

Evaluation spans five dimensions:

Resolution quality	correctness, completeness, evidence coverage, procedural accuracy, clarification quality, and escalation appropriateness.
Automation quality	safe auto-resolution rate, false automatic dispatch, policy violations, tool success, failure recovery, and reopening.
Reviewer leverage	review time, approval-without-edit rate, edit reasons, cases handled, specialist burden, and useful disagreement with AI.
Operational accountability	queue time, active handling time, customer wait, handoffs, evidence-ready time, override patterns, customer feedback, and final outcome.
Subsystem reliability	source-sync integrity, retrieval stability, model availability and quota, connector delivery, persistence, reconciliation, and recovery, attributed to the boundary that actually failed.

Evaluation must preserve this attribution. A request can retrieve the correct evidence while a model provider is unavailable; a model can draft a plausible answer while the evidence is incomplete; a correct approved response can still fail channel delivery. These are different failures with different owners and must not collapse into one generic “accuracy” or “system failed” label.

The rollout model should remain progressive:



The platform should support rollback at each stage. Changes to models, embeddings, retrieval, policies, knowledge versions, or tools must be evaluated against historical cases and observed after deployment.

Proof ladder

Proof of concept asks whether one bounded knowledge slice can complete the governed lifecycle with reproducible evidence, typed decisions, safe abstention, human review, and audit. **Proof of value** asks whether that system improves on the prior workflow using the same real cases and measured reviewer effort. A second materially different slice then tests whether the runtime is reusable rather than overfit to the first category.

14 MOHID as the First Reference Implementation

MOHID is the first implementation because it presents a concrete combination of favorable conditions:

- strong owner-authored support documentation;
- recurring support topics that can be grounded in approved sources;
- email-centered intake and existing escalation patterns;
- potential direct responses, ticket prefilling, form routing, and API actions;
- a need to reduce repetitive first-line work while preserving specialist authority; and
- a practical path to compare automatic, assisted, and human outcomes.

The first implementation should use the technology stack and deployment services approved by MOHID/DeenTek from the outset. Provider-neutral contracts and reusable adapter tests protect portability;

they do not justify building a separate temporary production stack or parallel integrations. Exact service choices and test gates belong in the companion MOHID living specification.

The strategic sequence is:

Prove one MOHID slice → prove value on the same real cases → add a different MOHID slice without rewriting the runtime → extract reusable primitives → repeat for a second organization

The implementation must remain narrow enough to complete. The vision becomes credible only when one end-to-end support workflow works reliably with real documentation, realistic stakeholder cases, meaningful human review, controlled tool execution, and a reconstructable audit trail. A deployed email loop proves transport; it does not by itself prove resolution quality or operational value.

15 Explicit Non-Goals

The platform is not intended to become:

- a generic chatbot whose success is measured by conversational fluency;
- a universal autonomous-agent operating system;
- a replacement for every CRM, help desk, or communication channel;
- an employee-surveillance or disciplinary scoring engine;
- a system that infers worker motive from delay or disagreement;
- an unrestricted no-code workflow builder, generic connector marketplace, or MCP platform for arbitrary business logic;
- an LLM-controlled transaction layer without deterministic authorization and recovery;
- a promise to eliminate human support roles;
- a multi-cloud platform with equal provider support before customer need exists; or
- a broad enterprise suite built before the first reference workflow is validated.

It may integrate with or eventually replace parts of these systems, but only when doing so directly improves the support-resolution lifecycle.

16 Closing Vision

The long-term opportunity is not merely to make support agents type faster or to place an AI chatbot in front of customers. It is to standardize how support requests are understood, evidenced, acted upon, reviewed, routed, and measured.

The platform should allow an organization to choose its own balance of automation and human judgment without sacrificing accountability. Routine, well-supported cases can be resolved immediately. Ambiguous or consequential cases can be prepared and routed to reliable reviewers. Managers can understand where cases stall and why. Reviewers can work from evidence instead of repeatedly searching from scratch. Automated actions can expand only when their performance is demonstrated.

Final product definition

The Governed Support Automation Platform is a configurable operating layer for support resolution. It turns approved knowledge into evidence-backed decisions, controls whether AI may act automatically, routes exceptions to the right humans, and preserves a complete record of how each case was resolved.

SON makes that process visible and controllable. The runtime makes it repeatable. The evidence and policy layers make it defensible. The case record makes both AI and human participation accountable.

One case. One governed lifecycle. One defensible outcome.

References

References

- [1] Zendesk, “Using auto assist to solve tickets,” Zendesk Help, updated July 17, 2026. <https://support.zendesk.com/hc/en-us/articles/7051314237466-Using-auto-assist-to-solve-tickets>
- [2] Intercom, “Fin AI Agent explained,” Intercom Help, updated June 25, 2026. <https://www.intercom.com/help/en/articles/7120684-fin-ai-agent-explained>
- [3] Intercom, “Hand over Fin AI Agent conversations to another support tool,” Intercom Help, updated June 12, 2026. <https://www.intercom.com/help/en/articles/7995955-hand-over-fin-ai-agent-conversations-to-another-support-tool>
- [4] Salesforce, “Get to Know Agentforce Service Agent,” Salesforce Help, accessed July 27, 2026. https://help.salesforce.com/s/articleView?id=service.bots_service_asa_about.htm&language=en_US&type=5
- [5] Model Context Protocol, “Specification 2025-11-25,” official specification, accessed July 27, 2026. <https://modelcontextprotocol.io/specification/2025-11-25>
- [6] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, January 2023. <https://doi.org/10.6028/NIST.AI.100-1>